What we have done, though, is created something that breaks a basic coding standard: PSR-1 stipulates (valid) UTF-8 encoding for all source code files. I chose a non-printing (extended control character) value of *0x80* to demonstrate that broken UTF-8 will still execute provided it satisfies the symbol naming rules; I could have chosen *0xBF*, which *is* a printable character in ISO-8859-1 – inverted question-mark ¿ – in fact, *0xA0* to *0xBF* are all printable in ISO-8859, so I highly recommend making sure any old PHP source code files you might inherit are correctly converted to UTF-8 (assuming you are working with PHP 7 or higher). I don't say this because PHP will have problems running the code; we've just demonstrated that PHP's only criterion for symbols is a valid sequence of bytes. Conversion is recommended because if everything is UTF-8 everywhere, you will ensure the entire toolchain (from IDEs and editors to continuous integration systems) will be frictionless, and any string literals that become output to other clients will be correctly rendered.

You are free to use UTF-8 liberally in your PHP programs; here are a couple of examples:

```
$∑ = 1 + 1; // ∑ (Greek sigma) denotes a summation operation

הפה' = $הפה'; // This snippet of Hebrew is a right-to-left string,
                directionality depends on your editor!
```

Note the example in Hebrew will very likely cause you some problems unless your system is correctly set up for the language and you type the characters in one at a time. PSR-1 specifies that all PHP source code files must be UTF-8 encoded, but that does not mean anyone is obliged to use a UTF-8-capable editor. The code above will execute just fine: as far as PHP is concerned, the variable $∑ is just a legal three-byte symbol (*0xE2 0x88 0x91*), but the file might not be readable to all users of your code. In the extended ASCII encoding of ISO-8859-1, the sequence *0xE2 0x88 0x91* maps to three codepoints: "â", plus two non-characters: a non-UTF-8 text editor is likely going to struggle with that.

In addition, the Hebrew characters used above are bidirectional: a fully Unicode-compliant editor will render them *before* the dollar symbol (even though they occur after the dollar symbol as the PHP parser reads it, which means they're legal). Not all editors,